CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE | DELHI NCR | PUNE

DEPARTMENT OF
COMPUTER SCIENCE AND ENGINEERING

L&T EduTech
Advanced JavaScript Frameworks (Angular)
CIA 2 and 3 – Projects Activities

Submitted by:

BAREEQ QAYOOM 2460343

SHREYA SUNIL MORAJKAR 2460450

JOHN JOBY C 2460380

NAVANEET NAIR 2460409

Employee Management Dashboard using Angular and TypeScript

Abstract

This report documents the development of a comprehensive Employee Management Dashboard built using the Angular framework with TypeScript. The application implements a modular, component-based architecture to deliver a responsive web interface for managing employee records. It demonstrates practical application of Angular concepts including components, services, routing, reactive forms, and integration with Angular Material UI components. The system features dynamic data rendering, simulated CRUD operations, and a professional user interface suitable for organizational human resource management.

Introduction

Modern web applications require robust, maintainable architectures to handle dynamic data management effectively. Angular, as a full-featured TypeScript-based framework, provides the necessary tools for building enterprise-grade single-page applications. This project develops an Employee Management Dashboard that serves as a practical implementation of Angular's core features. The application enables organizations to view, add, edit, and manage employee information through an intuitive interface, demonstrating real-world application of front-end development principles.

The dashboard showcases key Angular capabilities including dependency injection, data binding, component lifecycle management, and reactive programming with RxJS. By integrating Angular Material, the project achieves a polished, professional appearance while maintaining accessibility and responsiveness across devices.

Problem Statement

Organizations require efficient systems to manage employee information, including roles, departments, salaries, and contact details. Traditional spreadsheet-based management lacks real-time updates, validation, and user-friendly interfaces. There is a need for a web-based dashboard that provides:

1. Centralized Employee Data Management: A single source of truth for employee records
2. Real-time Data Interaction: Immediate updates without page refreshes
3. Data Validation: Ensured data integrity through form validation
4. Scalable Architecture: Modular structure allowing future feature additions
5. Professional User Interface: Intuitive design requiring minimal training
6. Cross-device Compatibility: Responsive design accessible on various screen sizes

Existing solutions often involve complex enterprise software with steep learning curves or custom-built applications lacking maintainability. This project addresses these gaps by developing a lightweight yet powerful Angular application that balances functionality with developer-friendly architecture.

Infrastructure Requirements

Development Environment:

- Operating System: Windows 10/11, macOS, or Linux
- Node.js: Version 18.x or higher (v22.19.0 used in implementation)
- npm: Version 9.x or higher (v11.6.2 used)
- Angular CLI: Version 17.x or higher (v21.0.0 used)
- IDE: Visual Studio Code with Angular extensions
- Version Control: Git for source code management

Project Dependencies:

- Core Framework: Angular v17+
- Language: TypeScript v5.x
- UI Library: Angular Material with Angular CDK
- Styling: CSS with optional SCSS/SASS
- HTTP Client: Angular HttpClient for API communication
- Reactive Programming: RxJS v7.x
- Forms: Angular Reactive Forms module
- Routing: Angular Router module

Development Tools:

- Package Manager: npm or yarn
- Build Tool: Angular CLI
- Code Quality: ESLint, Prettier
- Testing: Jasmine, Karma (unit testing), Protractor (e2e testing optional)
- API Simulation: JSON Server or Angular in-memory-web-api

Hardware Requirements:

- Processor: Dual-core 2.0 GHz or higher
- RAM: 8 GB minimum (16 GB recommended)
- Storage: 2 GB available space for development environment
- Network: Internet connection for package installation

System Architecture

Frontend Architecture:

The application follows Angular's component-based architecture with a clear separation of concerns:

```
Angular Application
├── Modules
│   ├── AppModule (root)
│   ├── MaterialModule (UI components)
│   └── SharedModule (reusable components)
├── Components
│   ├── EmployeeListComponent (displays employee table)
│   ├── EmployeeDetailComponent (shows individual employee)
│   ├── EmployeeFormComponent (add/edit employees)
│   └── NavbarComponent (navigation)
├── Services
│   └── EmployeeService (data operations)
├── Models
│   └── Employee (TypeScript interface)
├── Routing
│   └── AppRoutingModule (navigation configuration)
└── Directives/Pipes
    ├── DepartmentFilterPipe (custom filtering)
    └── HighlightDirective (DOM manipulation)
```

Data Flow:

- Data Retrieval: EmployeeService fetches data via HTTP client
- State Management: Components maintain local state with Input/Output bindings
- User Interaction: Event bindings capture user actions
- Data Updates: Forms submit validated data to service methods
- UI Updates: Angular change detection updates views automatically

Implementation Details

Employee Data Model:

typescript

```
export interface Employee {
  id: number;
  name: string;
  role: string;
  department: string;
  salary: number;
  email?: string;
  joinDate?: Date;
}
```

Core Components:

Employee List Component:

- Displays employees in a sortable, paginated table
- Uses *ngFor directive for dynamic rendering
- Includes action buttons (View, Edit, Delete)
- Implements data filtering and sorting

Employee Detail Component:

- Shows comprehensive employee information
- Uses route parameters to load specific employee data
- Implements lifecycle hooks (ngOnInit) for data loading

Employee Form Component:

- Built with Reactive Forms for complex validation
- Supports both create and update operations
- Includes form validation (required fields, pattern matching)
- Uses Angular Material form controls

Key Features Implemented:

- Angular Material Integration: MatTable, MatFormField, MatDialog components
- Routing with Parameters: Dynamic loading of employee details
- Reactive Forms: Template-driven and model-driven form approaches
- Custom Pipes: Department-based filtering and data formatting
- Directives: Custom attribute directives for UI enhancements
- HTTP Interceptors: For request/response handling and error management
- Dependency Injection: Service sharing across components

Technical Implementation

Routing Configuration:

typescript

```
const routes: Routes = [
  { path: '', redirectTo: '/employees', pathMatch: 'full' },
  { path: 'employees', component: EmployeeListComponent },
  { path: 'employees/:id', component: EmployeeDetailComponent },
  { path: 'add-employee', component: EmployeeFormComponent },
  { path: 'edit-employee/:id', component: EmployeeFormComponent }
];
```

Service Implementation:

The Employee Service utilizes Angular's Http Client to perform CRUD operations, returning Observables for asynchronous data handling. It implements methods for getAllEmployees(), getEmployeeById(), addEmployee(), updateEmployee(), and deleteEmployee().

Form Handling:

- Reactive forms with FormGroup and FormControl provide robust validation:
- Required field validation for name, role, department
- Salary validation with minimum and maximum values
- Email pattern validation using regular expressions
- Custom validators for business logic

Testing Strategy

- Unit Tests: Component testing with TestBed
- Service Tests: Mock HTTP requests using HttpClientTestingModule
- Form Tests: Validation logic testing
- Routing Tests: Navigation and parameter testing

Deployment Considerations

- Build Optimization: Production build with AOT compilation
- Asset Optimization: CSS/JS minification and bundling
- Environment Configuration: Separate configurations for development and production
- Hosting: Compatible with static hosting platforms (Firebase, Netlify, AWS S3)

Results and Output

Development Environment Setup

```
ng serve (employee-managen

Microsoft Windows [Version 10.0.26200.7462]
(c) Microsoft Corporation. All rights reserved.

C:\Users\johnj>node -v
v22.19.0

C:\Users\johnj>npm -v
11.6.2

C:\Users\johnj>ng version

Angular CLI       : 21.0.0
Node.js           : 22.19.0
Package Manager   : npm 11.6.2
Operating System  : win32 x64


C:\Users\johnj>cd "C:\Users\johnj\OneDrive\Desktop\Frontend\Advanced Java Script\Angular Project"

C:\Users\johnj\OneDrive\Desktop\Frontend\Advanced Java Script\Angular Project>dir
 Volume in drive C is Windows-SSD
 Volume Serial Number is A6D6-7C35

 Directory of C:\Users\johnj\OneDrive\Desktop\Frontend\Advanced Java Script\Angular Project

14-01-2026  09:44 AM    <DIR>          .
20-11-2025  02:32 PM    <DIR>          ..
14-01-2026  09:44 AM    <DIR>          Project 1
14-01-2026  09:44 AM    <DIR>          Project 2
               0 File(s)              0 bytes
               4 Dir(s)  130,475,642,880 bytes free

C:\Users\johnj\OneDrive\Desktop\Frontend\Advanced Java Script\Angular Project>cd "Project 1"

C:\Users\johnj\OneDrive\Desktop\Frontend\Advanced Java Script\Angular Project\Project 1>ng new employee-management-dashboard
```

Figure 1: Environment verification showing Node.js, npm, and Angular versions

Project Initialization

```
ng serve (employee-managen

C:\Users\johnj\OneDrive\Desktop\Frontend\Advanced Java Script\Angular Project\Project 1>ng new employee-management-dashboard
/ Which stylesheet system would you like to use? CSS            [
https://developer.mozilla.org/docs/Web/CSS                      ]
/ Do you want to enable Server-Side Rendering (SSR) and Static Site
Generation (SSG/Prerendering)? No
/ Which AI tools do you want to configure with Angular best practices?
https://angular.dev/ai/develop-with-ai None
CREATE employee-management-dashboard/angular.json (2045 bytes)
CREATE employee-management-dashboard/package.json (976 bytes)
CREATE employee-management-dashboard/README.md (1549 bytes)
CREATE employee-management-dashboard/tsconfig.json (990 bytes)
CREATE employee-management-dashboard/.editorconfig (331 bytes)
CREATE employee-management-dashboard/.gitignore (647 bytes)
CREATE employee-management-dashboard/tsconfig.app.json (444 bytes)
CREATE employee-management-dashboard/tsconfig.spec.json (456 bytes)
CREATE employee-management-dashboard/.vscode/extensions.json (134 bytes)
CREATE employee-management-dashboard/.vscode/launch.json (490 bytes)
CREATE employee-management-dashboard/.vscode/tasks.json (980 bytes)
CREATE employee-management-dashboard/src/main.ts (228 bytes)
CREATE employee-management-dashboard/src/index.html (326 bytes)
CREATE employee-management-dashboard/src/styles.css (81 bytes)
CREATE employee-management-dashboard/src/app/app.spec.ts (719 bytes)
CREATE employee-management-dashboard/src/app/app.ts (323 bytes)
CREATE employee-management-dashboard/src/app/app.css (0 bytes)
CREATE employee-management-dashboard/src/app/app.html (20464 bytes)
CREATE employee-management-dashboard/src/app/app.config.ts (324 bytes)
CREATE employee-management-dashboard/src/app/app.routes.ts (80 bytes)
CREATE employee-management-dashboard/public/favicon.ico (15086 bytes)
/ Packages installed successfully.
warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it
    Successfully initialized git.

C:\Users\johnj\OneDrive\Desktop\Frontend\Advanced Java Script\Angular Project\Project 1>cd employee-management-dashboard

C:\Users\johnj\OneDrive\Desktop\Frontend\Advanced Java Script\Angular Project\Project 1\employee-management-dashboard>ng serve
Initial chunk files | Names         | Raw size
main.js             | main          | 48.45 kB |
styles.css          | styles        | 95 bytes |

                    | Initial total | 48.55 kB

Application bundle generation complete. [4.508 seconds] - 2026-01-14T04:23:46.458Z
```

Figure 2: Angular CLI project creation process

Application Structure

```
ng serve (employee-managen
CREATE employee-management-dashboard/.gitignore (647 bytes)
CREATE employee-management-dashboard/tsconfig.app.json (444 bytes)
CREATE employee-management-dashboard/tsconfig.spec.json (456 bytes)
CREATE employee-management-dashboard/.vscode/extensions.json (134 bytes)
CREATE employee-management-dashboard/.vscode/launch.json (490 bytes)
CREATE employee-management-dashboard/.vscode/tasks.json (980 bytes)
CREATE employee-management-dashboard/src/main.ts (228 bytes)
CREATE employee-management-dashboard/src/index.html (326 bytes)
CREATE employee-management-dashboard/src/styles.css (81 bytes)
CREATE employee-management-dashboard/src/app/app.spec.ts (719 bytes)
CREATE employee-management-dashboard/src/app/app.ts (323 bytes)
CREATE employee-management-dashboard/src/app/app.css (0 bytes)
CREATE employee-management-dashboard/src/app/app.html (20464 bytes)
CREATE employee-management-dashboard/src/app/app.config.ts (324 bytes)
CREATE employee-management-dashboard/src/app/app.routes.ts (80 bytes)
CREATE employee-management-dashboard/public/favicon.ico (15086 bytes)
/ Packages installed successfully.
warning: in the working copy of 'package-lock.json', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'package.json', LF will be replaced by CRLF the next time Git touches it
    Successfully initialized git.

C:\Users\johnj\OneDrive\Desktop\Frontend\Advanced Java Script\Angular Project\Project 1>cd employee-management-dashboard

C:\Users\johnj\OneDrive\Desktop\Frontend\Advanced Java Script\Angular Project\Project 1\employee-management-dashboard>ng serve
Initial chunk files | Names         | Raw size
main.js             | main          | 48.45 kB |
styles.css          | styles        | 95 bytes |

                    | Initial total | 48.55 kB

Application bundle generation complete. [4.508 seconds] - 2026-01-14T04:23:46.458Z

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server per-request transformations.
  → Local:   http://localhost:4200/
  → press h + enter to show help
```

Figure 3: Project structure generation and development server startup

Functional Output

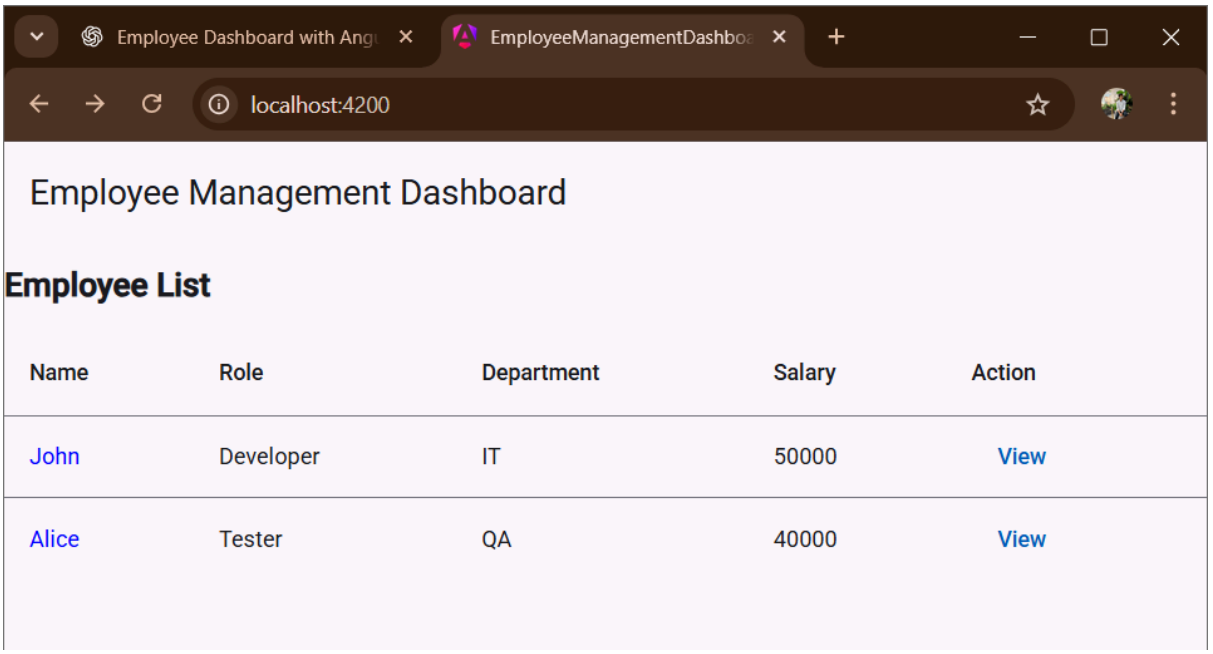

Figure 4: Employee list interface with sample data

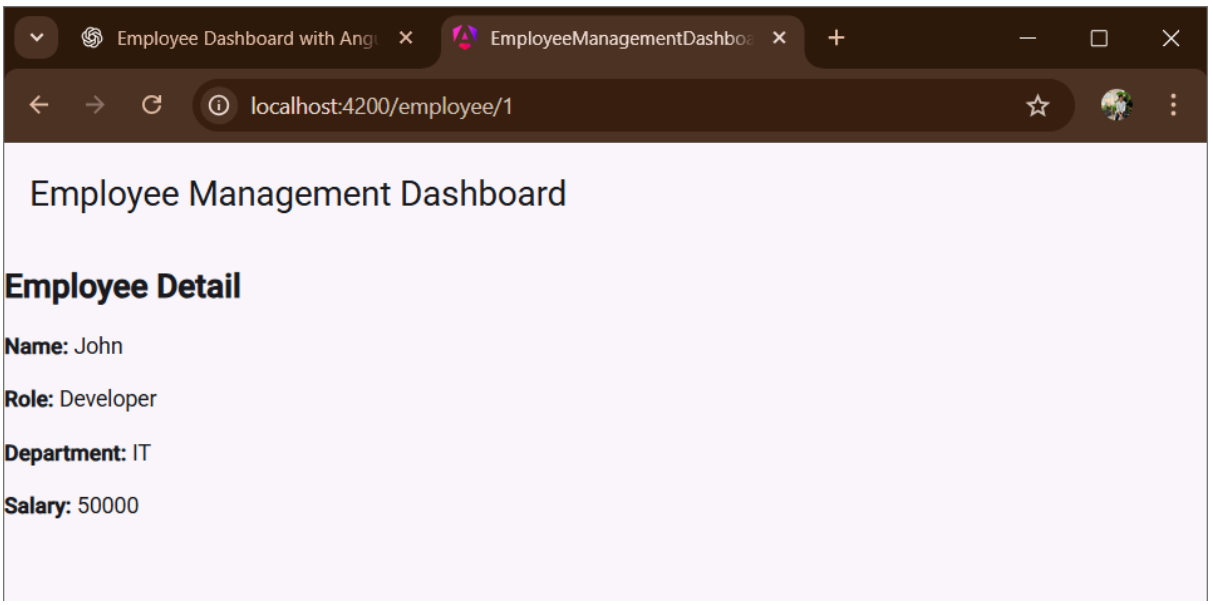

Figure 5: Employee detail view for John (Developer)

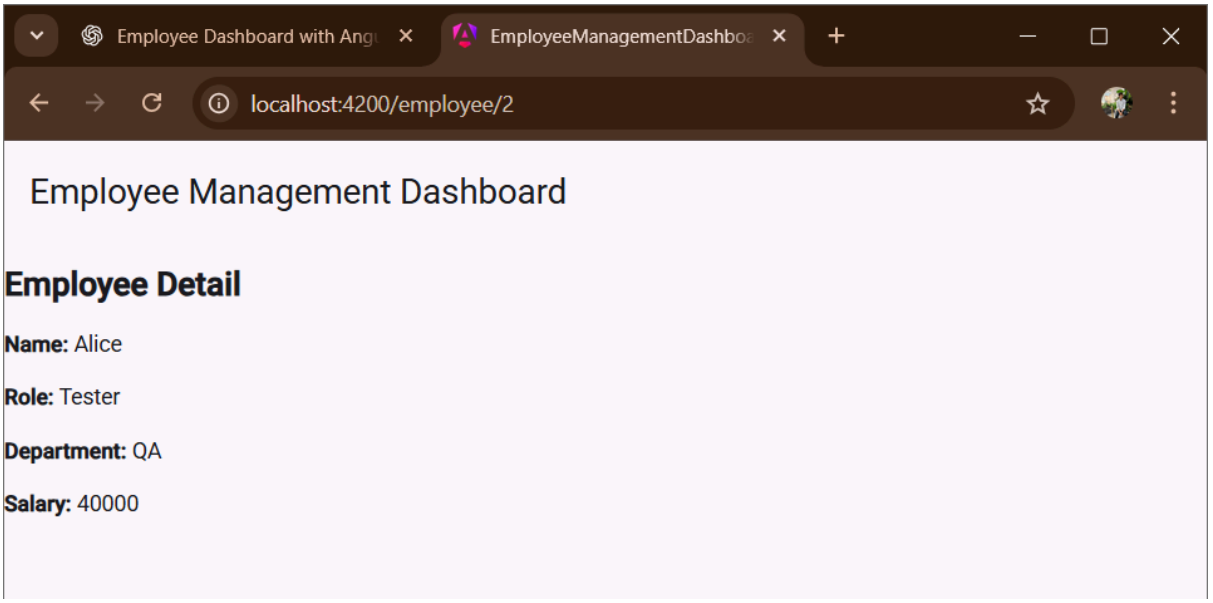

Figure 6: Employee detail view for Alice (Tester)

Conclusion

The Employee Management Dashboard successfully demonstrates the practical application of Angular framework capabilities for building enterprise web applications. The implementation showcases component-based architecture, reactive programming, and material design principles to create a maintainable, scalable solution for employee data management. The project serves as both a functional application and a learning resource for Angular development best practices, providing a foundation for further enhancements such as authentication, role-based access, and integration with backend APIs.